

Fourth assignment

Arrays and Strings

J. Sam & J.-C. Chappelier

1 Exercise 1 — Most Frequent Element in an Array

The purpose of this exercise is to write a program that identifies the most frequently occurring number in an array of integers. For example, for the following array :

```
{2, 7, 5, 6, 7, 1, 6, 2, 1, 7}
```

your program is supposed to find that the most occurring number is 7 and that its frequency is 3.

Download the source code `MostFrequent.java` available at the course webpage and complete it.

WARNING: you should modify neither the beginning nor the end of the program, only add your own lines as indicated. It is therefore very important to respect the following procedure (the points 1 and 3 concern only Eclipse users):

1. remove automated formatting of the code in Eclipse:

Window > Preferences > Java > Editor > Save Actions
(and untick the formatting option if it's on);

2. save the downloaded file as `MostFrequent.java` (respect the upper case). If you work with Eclipse, save it to `projectFolderUsedForThatExercise/src/`;
3. refresh the Eclipse project where the file is stored (right click on the project > "refresh") in order to take into account the new file;

4. write your code between these two provided comments:

```
/* *****  
 * Completez le programme a partir d'ici.  
 * *****/  
  
/* *****  
 * Ne rien modifier apres cette ligne.  
 * *****/
```

5. save and test your program to be sure that it works properly, for example using the values given below;
6. upload the modified file (still named `MostFrequent.java`) in "OUTPUT submission" (not in "Additional!").

Execution Example

It is mandatory that your code respects the following formatting:

```
Le nombre le plus frequent dans le tableau est le :  
7 (3 x)
```

There is a new line after the ":" symbol.

Your program must work for any array of integers given as input. You can however suppose that the arrays used for testing are always non-empty. Note that in some cases, in a given array there might be more than one number which occur the most. In such cases, your program must output the number that appears first in the array. E.g., for array `tab1 = {2, 7, 5, 6, 7, 1, 6, 2, 1, 7, 6}`, 6 and 7 are the most occurring numbers (both appear 3 times), but as 7 appears first in the array, the program will only keep 7 as the most frequently occurring element and will display:

```
Le nombre le plus frequent dans le tableau est le :  
7 (3 x)
```

2 Exercise 2 — Encryption

Julius Caesar was using a very simple encoding system, which consists of replacing every letter of a message by a letter placed several ranks after in the alphabet-

ical order. For example, for a shifting of 4, A becomes E, B becomes F, up to Z which becomes D.

Here we want to apply this technique to encode one string. For this you will write a program which applies the processing described below.

2.1 Encoding

The given program for this exercise provides you with a string `ALPHABET` containing all the letters of the latin alphabet, in lower case.

It asks the user to enter a string `s`.

The purpose of this exercise is to construct a new string `aCoder` as follows :

- `aCoder` contains, following their initial order, all the characters of `s` which are present in `ALPHABET` or that correspond to the space character (' ').
All the other characters are ignored.

The string `aCoder` will be encoded using the technique of Julius Caesar. **It has to be displayed.** If it does not contain any character, a message indicating that the string to encode is empty will be displayed.

As long as the string to encode is non-empty, you program should encode it with a fixed shifting given by the provided constant `DECALAGE`.

Encode the string `aCoder` by replacing every character with the character placed `DECALAGE` letters further in the string `ALPHABET`, and this in a circular way ('z' will be replaced by 'd' for example).

The spaces will be maintained as they are. They are not object of any encoding but will stay present at their position.

This encoded string will be displayed.

The execution of your program should strictly conform to the following examples :

```
Veillez entrer une chaine de caracteres :  
fuyez manants  
La chaine initiale etait : 'fuyez manants'  
La chaine a coder est : 'fuyez manants'  
La chaine codee est : 'jycid qererxw'
```

```
Veillez entrer une chaine de caracteres :  
avez-vous vu mes 3 chats et mes 2 chiens ?
```

```
La chaine initiale etait : 'avez-vous vu mes 3 chats et mes 2 chiens  
La chaine a coder est : 'avezvous vu mes  chats et mes  chiens '  
La chaine codee est : 'ezidzsyw zy qiw  glexw ix qiw  glmirw '
```

```
Veillez entrer une chaine de caracteres :  
93589()çç%&=+12AD  
La chaine initiale etait : '93589()çç%&=+12AD'  
La chaine a coder est vide.
```

The last message ("La chaine a coder est vide.") means "The string to be encoded is empty."

2.2 Procedure

Download the source code `Crypto.java` available at the course webpage and complete it.

WARNING: you should modify neither the beginning nor the end of the program, only add your own lines as indicated. It is therefore very important to respect the following procedure (the points 1 and 3 concern only Eclipse users):

1. remove automated formatting of the code in Eclipse:
Window > Preferences > Java > Editor > Save Actions
(and untick the formatting option if it's on);
2. save the downloaded file as `Crypto.java` (respect the upper case). If you work with Eclipse, save it to
`projectFolderUsedForThatExercise/src/`;
3. refresh the Eclipse project where the file is stored (right click on the project > "refresh") in order to take into account the new file;
4. write your code between these two provided comments:

```
/*  
 * Completez le programme a partir d'ici.  
 */  
  
/*  
 * Ne rien modifier apres cette ligne.  
 */
```

5. save and test your program to be sure that it works properly, for example using the values given below;
6. upload the modified file (still named `Crypto.java`) in "OUTPUT submission" (not in "Additional!").

3 Exercise 3 — Some "sense of ownership"

3.1 Introduction

A rich owner wants to close his land. You must write a program to help calculate the number of meters required for fence.

3.2 Description

Download the source code `Cloture.java` available at the course webpage and complete it.

WARNING: you should modify neither the beginning nor the end of the program, only add your own lines as indicated. It is therefore very important to respect the following procedure (the points 1 and 3 concern only Eclipse users):

1. remove automated formatting of the code in Eclipse:
Window > Preferences > Java > Editor > Save Actions
(and untick the formatting option if it's on);
2. save the downloaded file as `Cloture.java` (respect the upper case). If you work with Eclipse, save it to
`projectFolderUsedForThatExercise/src/`;
3. refresh the Eclipse project where the file is stored (right click on the project > "refresh") in order to take into account the new file;
4. write your code between these two provided comments:

```

/*****
 * Completez le programme a partir d'ici.
 *****/

/*****

```

```
* Ne rien modifier apres cette ligne.  
*****/
```

5. save and test your program to be sure that it works properly, for example using the values given below;
6. upload the modified file (still named `Cloture.java`) in "OUTPUT submission" (not in "Additional!").

3.3 Land Representation

The owner has a representation of his land in digitized form: the land is represented through a binary two-dimensional array. 1s are square plates that are part of his field and 0s are those that are not his land.

Thus, the task in this exercise is to calculate the number of meters of fence required to surround the land represented in this format. For this, it is necessary to count the number of 1s constituting the perimeter of the field. Each 1 on the perimeter corresponds to a 2.50 meters of fence for *each of its sides which are on the edge of the field*.

For example, the land

1

(all alone), which represents a tiny square terrain of 2.5 m on sides, requires 10 (= 4 times 2.5) meters of fence.

The land

0110
0110

needs 20 m of fence, and the land

0111110
0111110
0111110

40 m of fence.

The problem is a little bit more complex due to the fact that the field may contain ponds. The interior of the ponds is also represented by 0s but the edge of a pond should not be accounted in the perimeter of the field.

Example of a digitized terrain with ponds inside:

[illegible]

which would correspond to the following land:



For simplicity, we assume:

- that the land is “in one piece”: there exists no land in the map that is disconnected from each other;
- that there is no line that contains only 0s;
- that the outer perimeter of the field is “row convex”, meaning that for each line of the map of the land¹, the only 1s that belong to the outer perimeter are the first and the last ones on the line²; we can not have a line like this: “0011110001111” where 0s in the middle would be 0 outside the field; this kind of 0s necessarily represent a pond.

All this is to ensure that a 0 is inside the field (as part of a pond) if, on its line (but not necessarily on its column! See the example above.), there is at least a 1 before and at least a 1 after.

3.4 The code to be produced

The provided code contains messages to display in particular situations described below. You are required to use these lines as they are.

The code you are going to write will first verify that the provided array (`carte`) uniquely consists of 0s and 1s. If this is not the case, an error message *strictly* respecting the following format will be displayed (with line breaks):

```
Votre carte du terrain n'a pas le bon format :
valeur '2' trouvée en position [8][7]
```

The program stops in this case its execution. Note that the array we give in the provided code will be replaced by other arrays to test your code.

If the array `carte` representing the terrain is valid, your program will display the required number of meters of fence *strictly* respecting the display format given in the execution examples given below (including newline at end of message).

Indications:

- A simple way to proceed is to first “clear” the ponds (replacing the 0s of ponds by 1), then to proceed with the counting of 1s located on the perimeter.

¹However, we do not make this assumption for columns! See the example above.

²But there can be only one 1 when the first and the last 1s are the same: “000010000”

- One point of the perimeter can have multiple 0s as neighbors (for example, a 0 above and another 0 on the left), meaning that each of such sides must be fenced. It must in that case be counted as many times for the fence (twice in the example).
- To force the output of the `main` method, if you think that the execution of the program has to stop at some point, you can use the `return;` command.

3.5 Checking the row-convexity

Finally, we ask you to add, between checking that the card does contain only 0s and 1s and calculating the length of the fence, an additional check that the card is properly “row convex”.

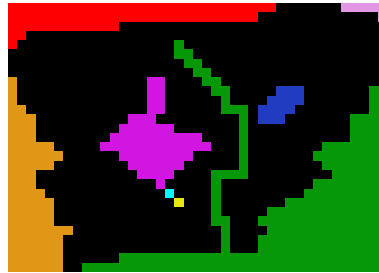
Note that this part, more difficult, is completely independent from the rest and you can completely calculate the length of the fence and have some points for this exercise, without having made this last part.

The requested verification will have to reject any map in which the 0s belonging to the exterior the private field are present between 1s of the same line, such as in this map:



The algorithm that we propose to detect these erroneous cards is:

1. find all zones of 0s (called “connected components”);
for example, different zones of 0s in the following image are represented by different colors:



2. find among these zones, those outside the private field (the others being the ponds);
3. browse the picture line by line to see if a zone of 0 outside the field is between two 1s.

Note that you can combine this last step with your “*clear*” *the ponds* step (described in the previous section).

In the first stage (finding the connected components), with programming knowledge at this stage of the course we propose to do the following:

1. declare two dynamic arrays of integers; these will be used to store the coordinates of points on the map during processing, one for ordinates (i.e., y-axis) `i`, one for the abscissas (i.e., x-axis) `j`;
2. declare an integer variable and initialize it with the value 1; we will use this variable to count and label the different zones of 0. For simplicity, let’s call it “`component`”; it will be increased by 1 for each new zone;
3. loop over all the positions of the map;
if the value at the current position is 0:
 - Increment the zone counter variable (`composante`) by 1;
 - Insert the ordinate `i` and the abscissa `j` of the current position to your dynamic arrays;
 - As these dynamic arrays are not empty:
 - retrieve and delete the values of the first element of each array (one abscissa and one ordinate);
 - if the value of the map (`carte`) at this newly retrieved position is 0:
 - * assign the value of the zone counter variable (`component`) to this position of the map;

- * for each of the neighbors of the recovered position (adjacent NORTH, SOUTH, EAST and WEST, if they exist): if the value of the neighbor is 0, add its coordinates (abscissa and ordinate) to your dynamic arrays.

If you print the previous map after this step, you should get:

[illegible]

For the second step, you just have to traverse the border of the map and store in a dynamic array all values encountered which are not 1. Although this is not optimal, it does not matter here to repeat several times the same value in this array.³

³With additional concepts presented later in the course, we could effectively add each value only one time, but it's not fundamental.

For the third and final step, just browse each line and look for any value that is not 1 between the first and the last 1 of this line. If such a value is present in the arrays constructed in the previous step (array of components of the boundary), then this means that the map is erroneous. The program must then display an error message *strictly* respecting the following format (with line breaks):

```
Votre carte du terrain n'a pas le bon format :  
bord extérieur entrant trouvé en position [4][18]
```

3.6 Execution examples

With the map given in the code and plotted above:

```
Il vous faut 385.0 mètres de clôture pour votre terrain.
```

With a map containing a 2 in position $i=8, j=7$:

```
Votre carte du terrain n'a pas le bon format :  
valeur '2' trouvée en position [8][7]
```

With the following plot:

```
01110  
01010  
01110
```

you should get:

```
Il vous faut 30.0 mètres de clôture pour votre terrain.
```

And with this one:

```
111  
001  
111
```

you should get:

```
Il vous faut 40.0 mètres de clôture pour votre terrain.
```

With a map not being “row convex” such as that shown above, you should get:

```
Votre carte du terrain n'a pas le bon format :  
bord extérieur entrant trouvé en position [4][18]
```